TDD mínimo para el cuestionario de entorno

Departamento de Informática

Índice

1 TDD mínimo para el cuestionario de entorno **1**
1.1 Qué es TDD . . . 1
1.2 Ciclo red-green-refactor . . . 1
1.2.1 Fase red . . . 1
1.2.2 Fase green . . . 2
1.2.3 Fase refactor . . . 2
1.3 Qué es un fixture . . . 2
1.4 JUnit básico . . . 2
1.5 Ejecutar pruebas con Maven Wrapper . . . 2
1.6 Spring Boot y servidor embebido . . . 3
1.7 Resumen para el cuestionario . . . 3
1.8 Comprobación rápida . . . 3

1 TDD mínimo para el cuestionario de entorno

Este documento explica solo lo necesario para responder al cuestionario **UD1 / Semana 02 — Entorno y TDD**. Más adelante se trabajará TDD con más profundidad en prácticas y APIs REST.

1.1 Qué es TDD

TDD significa **Test-Driven Development**, desarrollo guiado por pruebas.

La idea básica es escribir primero una prueba que describe un comportamiento esperado y después implementar el código necesario para que esa prueba pase.

No consiste en escribir pruebas al final para justificar que el código funciona. Eso puede ser útil, pero no es TDD.

1.2 Ciclo red-green-refactor

El ciclo habitual de TDD tiene tres pasos:

1. **Red**: escribir una prueba nueva y comprobar que falla.
2. **Green**: escribir la implementación mínima para que la prueba pase.
3. **Refactor**: mejorar el código sin cambiar el comportamiento observable.

1.2.1 Fase red

En la fase roja la prueba debe fallar por una razón útil: falta el comportamiento esperado.

Ejemplo: si se espera que `/health` responda correctamente y todavía no existe ese endpoint, la prueba puede fallar con `404 Not Found`.

Eso no es un fracaso: es una señal de que la prueba ha detectado la diferencia entre lo que esperamos y lo que existe ahora.

1.2.2 Fase green

En la fase verde no se intenta construir toda la aplicación perfecta. Se escribe el cambio mínimo que hace pasar la prueba.

Esto ayuda a avanzar por pasos pequeños y verificables.

1.2.3 Fase refactor

Refactorizar es mejorar la estructura del código sin cambiar lo que hace desde fuera.

Durante esta fase las pruebas deben seguir pasando. Si al refactorizar cambian los resultados, ya no estamos solo ordenando el código: hemos cambiado comportamiento.

1.3 Qué es un fixture

Un **fixture** es el estado inicial que necesita una prueba para ejecutarse de forma controlada.

Puede incluir:

- objetos preparados;
- datos de ejemplo;
- configuración concreta;
- una base de datos en un estado conocido;
- un servidor arrancado para pruebas.

La idea importante es que cada prueba debe poder ejecutarse de forma independiente. Compartir y mutar el mismo fixture global entre pruebas suele ser mala idea porque una prueba puede dejar restos que afecten a otra.

1.4 JUnit básico

JUnit es una herramienta para escribir y ejecutar pruebas automatizadas en Java.

Un método se marca como prueba con `@Test`:

```
import org.junit.jupiter.api.Test;

class ExampleTest {

    @Test
    void shouldAddTwoNumbers() {
        // prueba
    }
}
```

Una aserción comprueba un resultado esperado:

```
assertEquals(expected, actual);
```

Si `actual` no coincide con `expected`, la prueba falla.

1.5 Ejecutar pruebas con Maven Wrapper

El **Maven Wrapper** permite ejecutar Maven con la versión fijada por el proyecto. Así no dependemos de que cada equipo tenga instalada la misma versión global de Maven.

En Linux o macOS:

```
./mvnw test
```

En Windows:

```
mvnw.cmd test
```

Para que el entorno sea reproducible, el proyecto debe fijar también la versión de Java. En esta unidad se trabaja con **Java 25**.

1.6 Spring Boot y servidor embebido

Una aplicación Spring Boot suele tener una clase principal marcada con:

```
@SpringBootApplication
```

Cuando se arranca una aplicación web Spring Boot, el servidor HTTP embebido se ejecuta dentro del mismo proceso de la aplicación. No hace falta instalar un servidor externo para probar el ejemplo inicial.

En el servidor mínimo de UD1, el endpoint de comprobación de salud es:

```
GET /health
```

1.7 Resumen para el cuestionario

- **Maven Wrapper**: ejecuta Maven con la versión fijada por el proyecto.
- **Spring Boot**: simplifica la configuración y el arranque de una aplicación Spring.
- **JUnit**: permite escribir y ejecutar pruebas automatizadas en Java.
- **Fixture**: prepara datos y condiciones iniciales conocidas para una prueba.
- **Red**: la prueba falla porque falta el comportamiento esperado.
- **Green**: se implementa lo mínimo para que la prueba pase.
- **Refactor**: se mejora el código manteniendo las pruebas en verde.
- `@Test`: marca un método como prueba en JUnit 5.
- `assertEquals(expected, actual)`: comprueba que el valor real coincide con el esperado.
- `./mvnw test`: ejecuta las pruebas del proyecto en Linux/macOS.
- `@SpringBootApplication`: identifica habitualmente la clase principal de una aplicación Spring Boot.
- `GET /health`: endpoint de salud del servidor mínimo de UD1.

1.8 Comprobación rápida

Antes del cuestionario deberías poder explicar:

1. Por qué en TDD no se escribe primero toda la implementación.
2. Qué significa que una prueba esté en rojo.
3. Qué se puede cambiar durante refactorización y qué no.
4. Para qué sirve un fixture.
5. Por qué `./mvnw test` es más reproducible que depender del Maven global del sistema.